



## Department of Artificial Intelligence and Machine Learning

### Experiment No.: 2

#### To perform and implement Hiring Problem..

### Aim

To perform and implement the Hiring Problem algorithm and analyze its performance.

### Learning Objectives

After completing this experiment, students will be able to:

- Understand and implement the Hiring Problem algorithm.
- Explain the concept of probabilistic analysis.
- Understand the importance of randomized algorithms in optimization problems.

### Theory

The **Hiring Problem** is a classical algorithm used to explain the concepts of probabilistic analysis and randomized algorithms. A company interviews  $n$  candidates one by one in a random order. After each interview, an immediate decision must be made whether to hire the candidate. The hiring strategy is as follows:

- Hire the first candidate.
- For every subsequent candidate:
  - Compare the candidate with the currently hired candidate.
  - If the new candidate is better, hire the candidate and replace the current employee.
  - Otherwise, reject the candidate.

The objective is to ensure that the best candidate is hired while minimizing the number of hiring decisions.



## Algorithm

1. Read the list of candidates.
2. Hire the first candidate.
3. Assume the first candidate as the current best.
4. Traverse the remaining candidates.
5. Compare each candidate with the current best.
6. If the current candidate is better:
  - Hire the candidate.
  - Update the current best.
7. Display the best candidate and the total number of hiring decisions.

## Code and Output

```
hiring.py > ...
1  from itertools import permutations
2  import random
3  def get_permutations(candidate):
4      random_candidates = list(permutations(candidate))[0]
5      print("Candidates: ",random_candidates)
6      return random_candidates
7
8
9  best = 0
10 # current candidates
11 candidates = [random.randint(1,11) for _ in range(10)]
12 # randomized
13 randomized_candidates = get_permutations(candidates)
14 best_candidates = []
15 count_hired = 0
16 for i in range(len(candidates)):
17     if randomized_candidates[i] > best:
18         best = randomized_candidates[i]
19         best_candidates.append(best)
20         count_hired += 1
21
22 print("Hired Candidates: ",best_candidates)
23 print("Total Hired: ",count_hired)
24 print("Total Fired: ",count_hired - 1)
25
```

```
Candidates: (5, 10, 9, 7, 2, 10, 7, 10, 5, 10)
Hired Candidates: [5, 10]
Total Hired: 2
Total Fired: 1
```

## Problem Statement

1. There are  $n$  workers. You are given two integer arrays `quality` and `wage` where `quality[i]` is the quality of the  $i$ th worker and `wage[i]` is the minimum wage expectation for the  $i$ th worker. We want to hire exactly  $k$  workers to form a paid group. To hire a group of  $k$  workers, we must pay them according to the following rules:

- Every worker in the paid group must be paid at least their minimum wage expectation.
- In the group, each worker's pay must be directly proportional to their quality. This means if a worker's quality is double that of another worker in the group, then they must be paid twice as much as the other worker.

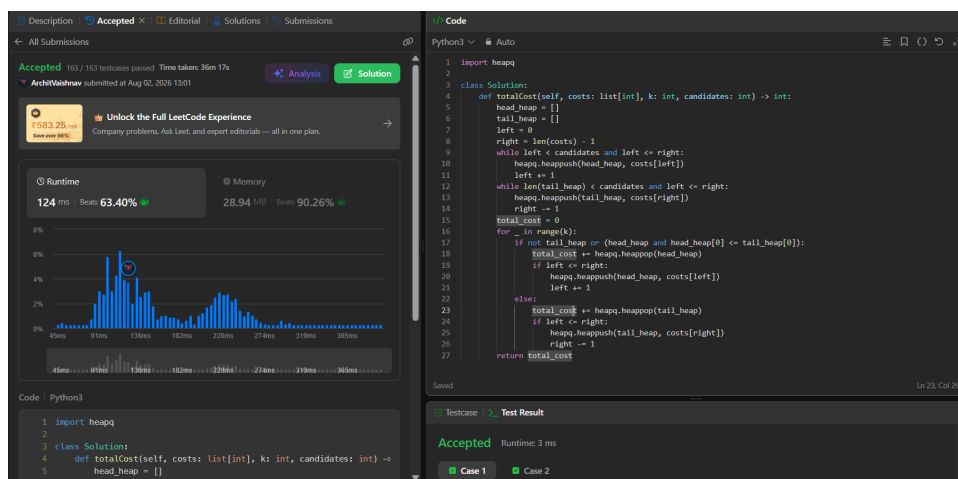
Given the integer  $k$ , return the least amount of money needed to form a paid group satisfying the above conditions. Answers within  $10^{-5}$  of the actual answer will be accepted.

### Example 1:

**Input:** `quality = [10,20,5], wage = [70,50,30], k = 2`  
**Output:** 105.00000  
**Explanation:** We pay 70 to 0<sup>th</sup> worker and 35 to 2<sup>nd</sup> worker.

### Example 2:

**Input:** `quality = [3,1,10,10,1], wage = [4,8,2,2,7], k = 3`  
**Output:** 30.66667  
**Explanation:** We pay 4 to 0<sup>th</sup> worker, 13.33333 to 2<sup>nd</sup> and 3<sup>rd</sup> workers separately.





2. You are given a **0-indexed** integer array `costs` where `costs[i]` is the cost of hiring the  $i^{th}$  worker. You are also given two integers `k` and `candidates`. We want to hire exactly `k` workers according to the following rules:

- You will run `k` sessions and hire exactly one worker in each session.
- In each hiring session, choose the worker with the lowest cost from either the first `candidates` workers or the last `candidates` workers. Break the tie by the smallest index.
  - For example, if `costs` = [3,2,7,7,1,2] and `candidates` = 2, then in the first hiring session, we will choose the 4<sup>th</sup> worker because they have the lowest cost [3,2,7,7,1,2].
  - In the second hiring session, we will choose the 1<sup>st</sup> worker because they have the same lowest cost as the 4<sup>th</sup> worker, but they have the smaller index [3,2,7,2]. Please note that the indexing may be changed in the process.
- If there are fewer than `candidates` workers remaining, choose the worker with the lowest cost among them. Break the tie by the smallest index.
- A worker can only be chosen once.

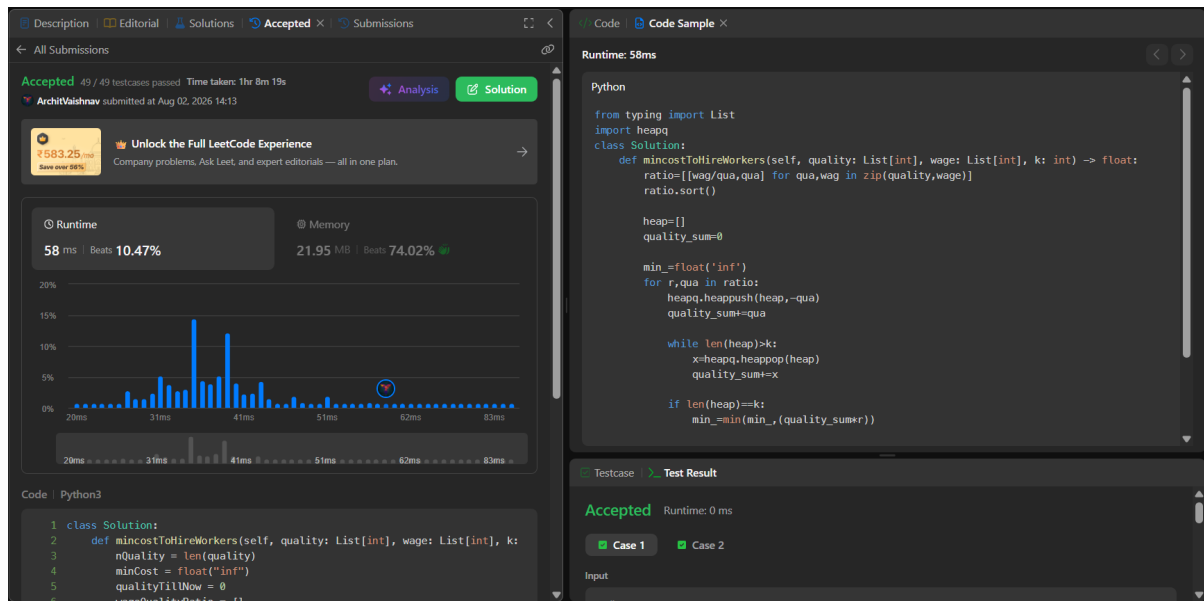
Return the *total cost to hire exactly k workers*.

**Example 1:**

**Input:** `costs` = [17,12,10,2,7,2,11,20,8], `k` = 3, `candidates` = 4  
**Output:** 11  
**Explanation:** We hire 3 workers in total. The total cost is initially 0.  
 – In the first hiring round we choose the worker from [17,12,10,2,7,2,11,20,8]. The lowest cost is 2, and we break the tie by the smallest index, which is 3. The total cost = 0 + 2 = 2.  
 – In the second hiring round we choose the worker from [17,12,10,7,2,11,20,8]. The lowest cost is 2 (index 4). The total cost = 2 + 2 = 4.  
 – In the third hiring round we choose the worker from [17,12,10,7,11,20,8]. The lowest cost is 7 (index 3). The total cost = 4 + 7 = 11. Notice that the worker with index 3 was common in the first and last four workers.  
 The total hiring cost is 11.

**Example 2:**

**Input:** `costs` = [1,2,4,1], `k` = 3, `candidates` = 3  
**Output:** 4  
**Explanation:** We hire 3 workers in total. The total cost is initially 0.  
 – In the first hiring round we choose the worker from [1,2,4,1]. The lowest cost is 1, and we break the tie by the smallest index, which is 0. The total cost = 0 + 1 = 1. Notice that workers with index 1 and 2 are common in the first and last 3 workers.  
 – In the second hiring round we choose the worker from [2,4,1]. The lowest cost is 1 (index 2). The total cost = 1 + 1 = 2.  
 – In the third hiring round there are less than three candidates. We choose the worker from the remaining workers [2,4]. The lowest cost is 2 (index 0). The total cost = 2 + 2 = 4.  
 The total hiring cost is 4.



## Viva Questions

1. What is the Hiring Problem?
2. Why is the Hiring Problem considered a probabilistic algorithm?
3. What is the objective of the Hiring Problem?
4. What is meant by a hiring cost?
5. What is the difference between interview cost and hiring cost?
6. What is the expected number of hires if the candidates arrive in random order?
7. Why is randomization useful in the Hiring Problem?
8. What is the worst-case scenario for the Hiring Problem?
9. What is the best-case scenario for the Hiring Problem?

## References

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, 4th Edition, MIT Press, 2022.
2. Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, *The Design and Analysis of Computer Algorithms*, Pearson Education.
3. MIT OpenCourseWare, *Introduction to Algorithms*. Available at: <https://ocw.mit.edu>